Architecture of a DBMS: PostgreSQL

Derrick Munyiri

Nairobi Linux Users Group Meetup

April 2026

Introduction

In this presentation, we learn about the architecture of Database Management Systems, with a focus on PostgreSQL. Why PostgreSQL?

- Postgres is powerful and reliable.
- It is widely used and time-tested.
- It is open source.

Overview

DBMS systems are composed of 5 main components:

- **Client communications manager** - Establish and remember the connection state for the caller, to respond to SQL commands from the caller, and to return both data and control messages (result codes, errors, etc.) as appropriate.
- **Process Manager** - A thread of computation must be assigned to the command/query. Also determines whether processing should begin immediately or deferred when resources are available.
- **Relational Query Processor** - Set of modules that parse the query, compiles it into an internal query plan, and executes it. Other actions include permission checks i.e whether the user is allowed to run the query.
- **Transaction Storage Manager** - Manages data access and manipulation calls. The storage system also includes algorithms and data structures for organizing data on disk. Buffer mechanisms between disk and memory are also handled here.
- **Shared Components and Utilities** - These are utilities which are invoked during or independent of any transactions. They ensure full functioning of a DBMS. Memory manager, for example, is used whenever memory needs to be dynamically allocated or dispersed.

Overview

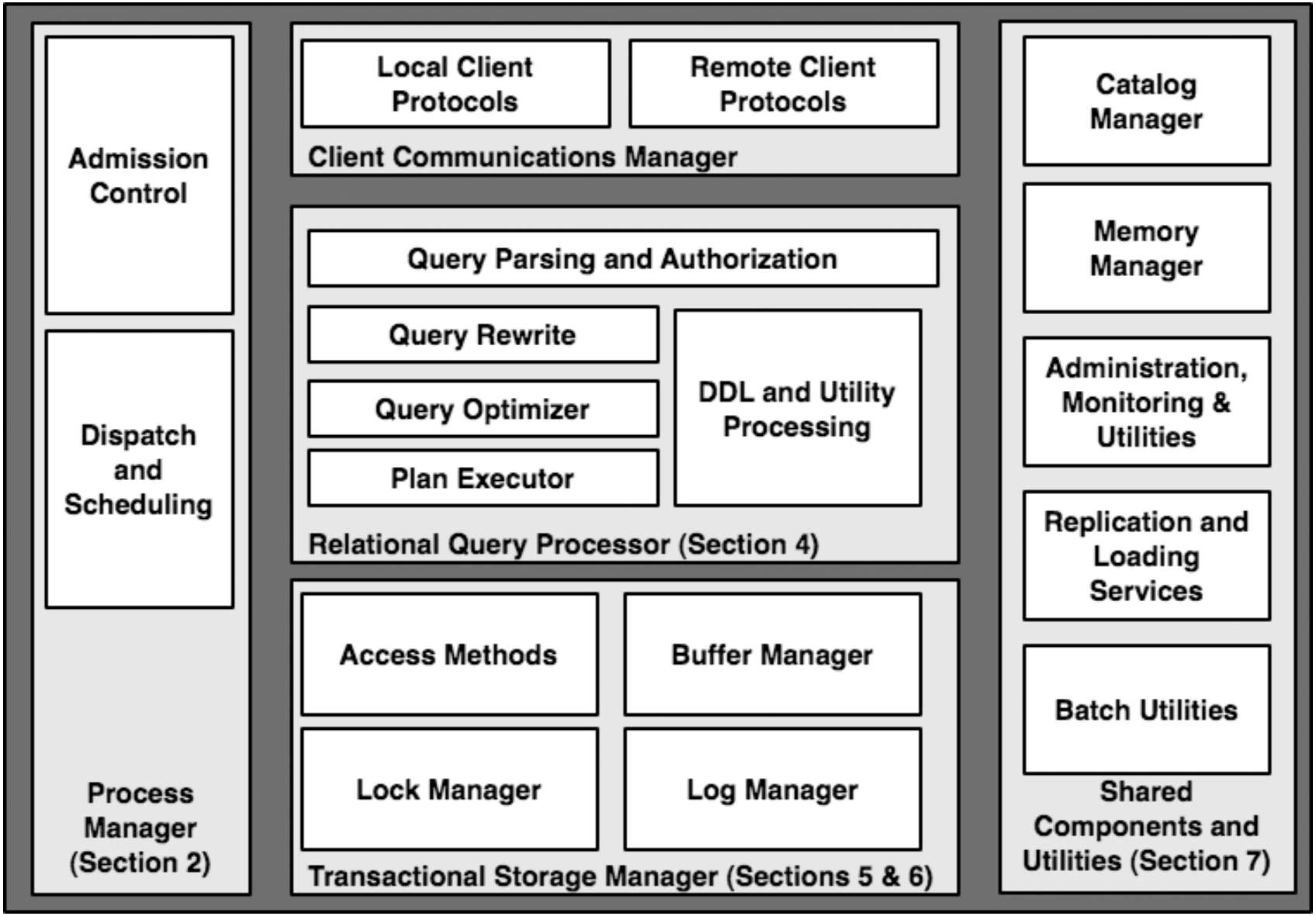

Figure: Main components of a DBMS.

Client Communications Manager

- PostgreSQL uses a message-based protocol for communication between frontend and backend.
- The protocol is supported over TCP/IP and Unix-domain sockets.
- In the startup phase, the frontend opens a connection and authenticates to the server.
- During normal operation, the frontend sends queries and commands; the backend returns results and responses.
- Session termination is usually initiated by the frontend, though the backend can force a close.
- `libpq` is the client library that implements this wire protocol (used by `psql` and other frontends).

Process Manager

- Once a client establishes a connection, a DBMS worker does work on behalf of that client.
- Each client has its own worker — a 1:1 ratio between client and worker. The worker handles all SQL requests.
- A new OS process is forked and becomes the backend for that connection.
- This process model is called the **process per worker model**.
- Relatively easy to implement: the OS scheduler manages timesharing of DBMS workers, and the system benefits from OS protection facilities (e.g., protection from memory overruns).

Process Manager

- **Drawback:** Processes need to share data structures (e.g., global lock table, buffer pool) across connections.
- **Solution:** Explicitly allocate these shared structures in OS-supported memory. Every DBMS worker process is made aware of the shared memory region at startup.
- Processes consume more memory than threads, and context switching is expensive $\rightarrow$ poor scaling under many concurrent connections.
- **Connection poolers (e.g., PgBouncer)** fix this overhead by maintaining a small pool of persistent connections and routing requests through them, eliminating the need to spin up new processes per client.

Relational Query Processor

- The relational query processor takes a declarative SQL statement, validates it, optimizes it into a procedural dataflow execution plan, and executes that plan (subject to admission control) on behalf of a client.
- **Parser stage:**
  - Validates query syntax using grammar and lexer files.
  - Generates a parse tree (machine-understandable format) if valid.
  - Performs semantic interpretation: resolves table names, column types, functions, and operators using system catalogs.
- **Query rewrite rule system:**
  - Rewrites and normalizes the query without changing semantics.
  - Applies rules (e.g., expands views into underlying tables, applies custom rewrite rules).
  - Passes the modified query to the planner/optimizer.
  - Example: When you query a view, this step replaces the view reference with the actual view definition.

Relational Query Processor

- **Planner/Optimizer:**
  - Creates an optimal execution plan from many possible ways to execute a query.
  - Examines feasible alternatives and selects the plan expected to run fastest.
  - Output is a **tree of plan nodes**.
- **Types of plan nodes:**
  - Scan nodes (SeqScan, IndexScan, BitmapHeapScan)
  - Join nodes (NestedLoop, HashJoin, MergeJoin)
  - Auxiliary nodes (Sort, Aggregate, Limit)
- **EXPLAIN statement:**
  - Displays the query plan: referenced tables, scan methods, and join algorithms (for multiple tables).
  - `EXPLAIN ANALYZE` actually executes the query and shows runtime statistics.

Relational Query Processor

- **Executor:**
  - Takes the plan from the planner/optimizer and recursively processes it to extract rows.
  - Implements a **demand-pull pipeline mechanism**: each time a plan node is called, it delivers one more row or reports completion.
- Each node in the plan is an operator.
- Each operator:
  - Receives tuples, processes them, and emits them.
  - Exposes a function like `getnext()` to fetch the next tuple.
- This is called the **iterator model** or **Volcano model**.
- Iterators combine dataflow with control flow:
  - `get_next()` returns a tuple reference via the call stack and returns control.
  - Only one process is needed for sequential execution.

Relational Query Processor

- **Parallel query execution:**
  - Each worker runs its own iterator pipeline.
  - The leader process pulls results from workers.
  - Special plan nodes: `GATHER` and `GATHER MERGE` (each has exactly one child plan — the portion executed in parallel).

Relational Query Processor

- **Memory allocation**
  - **How are tuples stored in memory and passed between operators?**
  - **Tuple descriptors:** An array of column references, each having a reference to the tuple in memory and the column offset within that tuple. Each iterator is pre-allocated a fixed number of these.
  - **Buffer pool:** Where actual tuples are stored.
  - **work_mem:** A configuration setting that determines memory allocated per operation before spilling to temporary disk files. Used for sorts, hash tables, and aggregates.
    - If multiple sort/hash operations run concurrently across many transactions, setting `work_mem` too high can cause memory exhaustion.

Transactional Storage Manager

- **Storage Management**
  - **Spatial control (controlling storage in space):** PostgreSQL uses standard OS file system utilities. It creates a file in the OS file system and manages positioning of data and offsets within that file.
  - The OS filesystem sits between applications and the disk, mapping high-level operations to low-level disk block reads and writes on the physical device.
  - Most favoured filesystems for PostgreSQL are EXT4 and XFS:
    - EXT4: simplicity and stability.
    - XFS: high performance, especially for large-scale workloads, but requires significant tuning.

Transactional Storage Manager

- **Storage Management**
  - **Temporal Control:** The DBMS controls when data is physically written to disk, with logic to handle block write timing.
  - **Buffer pool (shared_buffers):** Loads disk pages, uses them directly in native format, and later writes them back to disk.
  - **Log tail (WAL buffer):** An in-memory queue storing generated log entries, corresponding to the WAL file on disk. Periodically flushed to disk.
  - **Commit transaction flush:** A commit request waits until the commit log is flushed to disk before returning success $\rightarrow$ guarantees durability.
  - **WAL benefit:** Significantly reduces disk writes because only the WAL file needs to be flushed to guarantee a transaction is committed, rather than every data file changed by the transaction.

Transactional Storage Manager

- **Concurrency Control and Recovery**
  - PostgreSQL is an ACID-compliant database.
  - **Atomicity:** The "all or nothing" guarantee — either all of a transaction's actions commit or none do.
  - **Consistency:** Given a set of constraints, the DB can only commit if it leaves the database in a consistent state.
  - **Isolation:** Two concurrent transactions will not see each other's uncommitted updates.
  - **Durability:** Updates of a committed transaction will be visible to subsequent transactions, independent of subsequent hardware or software errors.
  - **Serializability:** Relates to the correctness of concurrent transactions (or a series of multiple committing transactions).

Transactional Storage Manager

- **Multi-Version Concurrency Control (MVCC)**
  - The mechanism PostgreSQL employs to enforce serializability.
  - Transactions do not hold locks, but are guaranteed a consistent view of the database state at some time in the past, even if rows have changed since that fixed point in time.
  - Uses multiple versions of the same logical table row to ensure clients always have a consistent (albeit potentially different) view of data.
  - Locks acquired for reading don't conflict with locks for writing $\rightarrow$ reading never blocks writing, and writing never blocks reading.
  - Rows are not modified in-place: for an update, the original row is marked as obsolete, and a new row is created.
  - Cleanup/garbage collection is needed regularly to remove stale row versions.
  - **Key takeaway:** Reads don't block writes; writes don't block reads.

Transactional Storage Manager

- **Anomalies**
  - **Dirty read:** Transaction reads data written by a concurrent uncommitted transaction.
  - **Non-repeatable read:** Transaction re-reads previously read data and finds it modified by another transaction that committed since the initial read.
  - **Phantom read:** Transaction re-executes a query returning a set of rows satisfying a condition and finds the set changed due to another recently committed transaction.
- **PostgreSQL isolation levels**
  - **Read committed**
    - Reads can only see committed data **before** the query began.
    - Does not see in-flight data or data committed by concurrent transactions **during** query execution.
    - Two successive `SELECT` statements can see different data, even within the same transaction, if other transactions commit changes between the two reads.

Transactional Storage Manager

- **Serializable**
  - The strictest isolation level. Emulates serial transaction execution.
  - Applications must be prepared to retry transactions once they are aborted.
  - `SELECT` queries see only data committed before the transaction began — never uncommitted data or changes committed by concurrent transactions during execution.
  - If an `UPDATE` (or `DELETE` or `SELECT FOR UPDATE`) targets a row already updated by a concurrent uncommitted transaction:
    - The second transaction waits for the first to commit or rollback.
    - If the first rolls back, the waiting transaction proceeds to change the row.
    - If the first commits, the serializable transaction is rolled back.
  - Under the hood, serializable uses **Serializable Snapshot Isolation (SSI)**.

Shared Components

- **System Catalogs**
  - The system catalogs are the place where a relational database management system stores schema metadata, such as information about tables and columns, and internal bookkeeping information.
  - PostgreSQL's system catalogs are regular tables.
  - Normally, one should not change the system catalogs by hand.
  - Can be accessed using pg_catalogs.
- **Memory Manager**
  - PostgreSQL allocates memory within memory contexts, which provide a convenient method of managing allocations made in many different places that need to live for differing amounts of time.
  - A memory context is an in-memory data structure that maintains a list of regions of contiguous virtual memory, often called memory pools.